

Docker Fundamentals

For the Modular Monolith

Build. Run. Inspect. Debug. Clean.

Repo: `scheduling-app` · Image: `scheduler:latest`

Why Docker Exists (and What It Is Not)

- **Problem it solves:** “Works on my machine” → consistent, reproducible environments
- **What it is:** packaging + runtime for containers
- **What it is NOT:** a full virtual machine (containers share the host kernel)
- **Why teams use it:** faster builds, simpler deploys, predictable runtime

Scalability connection:

- same image → many identical containers
- easy to scale horizontally with orchestration later (Compose/Kubernetes)

Learning Goals

By the end, you can:

- Explain Docker vs image vs container
- Build this repo into an image
- Run the container with ports + env
- Inspect state, logs, processes
- Debug a failing container
- Clean up images/containers safely

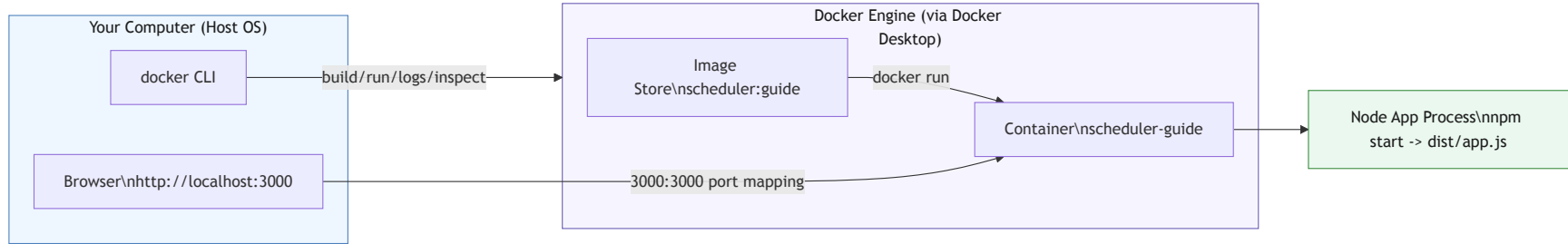
What Docker Is (Working Model)

- **Docker:** tooling + runtime for containers
- **Image:** immutable blueprint (filesystem + metadata)
- **Container:** running instance of an image
- **Dockerfile:** recipe to build an image
- **Registry:** remote image store (Docker Hub)

Key mental split:

- **Image is not running**
- **Container is running**

Mental Model Diagram



What to notice:

- CLI talks to Docker Engine, not directly to the app
- Images are stored; containers are running processes
- Port mapping bridges host traffic to the container

Key Definitions

- **Docker:** build/run containers
- **Image:** immutable packaged blueprint
- **Container:** running instance
- **Dockerfile:** build recipe
- **Registry:** remote image store
- **Port mapping:** host → container

Common Confusion

- Image ≠ running
- Container = running
- Same image → many containers

Examples:

- `scheduler:guide` (image)
- `scheduler-guide` (container)

This Repo's Docker Setup

```
1 FROM node:24.13.0-alpine3.23
2 WORKDIR /app
3 COPY package.json package-lock.json ./
4 RUN npm ci
5 COPY . .
6 RUN npm run build
7 EXPOSE 3000
8 CMD ["npm", "start"]
```

Key files: `Dockerfile` , `package.json` , `.env`

CMD starts the app with `npm start` .

Default command for the container; can be overridden at runtime. [CMD docs](#)

Why Copy package*.json First?

Docker builds in layers.

If dependencies don't change, the install step stays cached and rebuilds are faster.

Why it matters:

- **Fast feedback** during development
- **Smaller rebuild scope** when only code changes

```
1 Layer 1: FROM node:24.13.0-alpine3.23
2 Layer 2: COPY package*.json
3 Layer 3: RUN npm ci   ✅ cached if lockfile unchanged
4 Layer 4: COPY src
5 Layer 5: RUN npm run build
```


Verify Docker Is Ready

Run:

```
1  docker --version
2  docker context ls
3  docker info
```

What you're checking:

- CLI exists (`--version`)
- active engine target (`context ls`)
- daemon reachable (`info`)

If `docker info` fails, Docker Desktop (daemon) is not running yet.

Demo: Docker Ready Check

This recording shows the three sanity checks: CLI, context, daemon.

```
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
bash-3.2$
```

Build the Image

From repo root:

```
1  docker build -t scheduler:guide .
```

What it does:

- Reads `Dockerfile`
- Sends build context (`.`)
- Produces image `scheduler:guide`

Important:

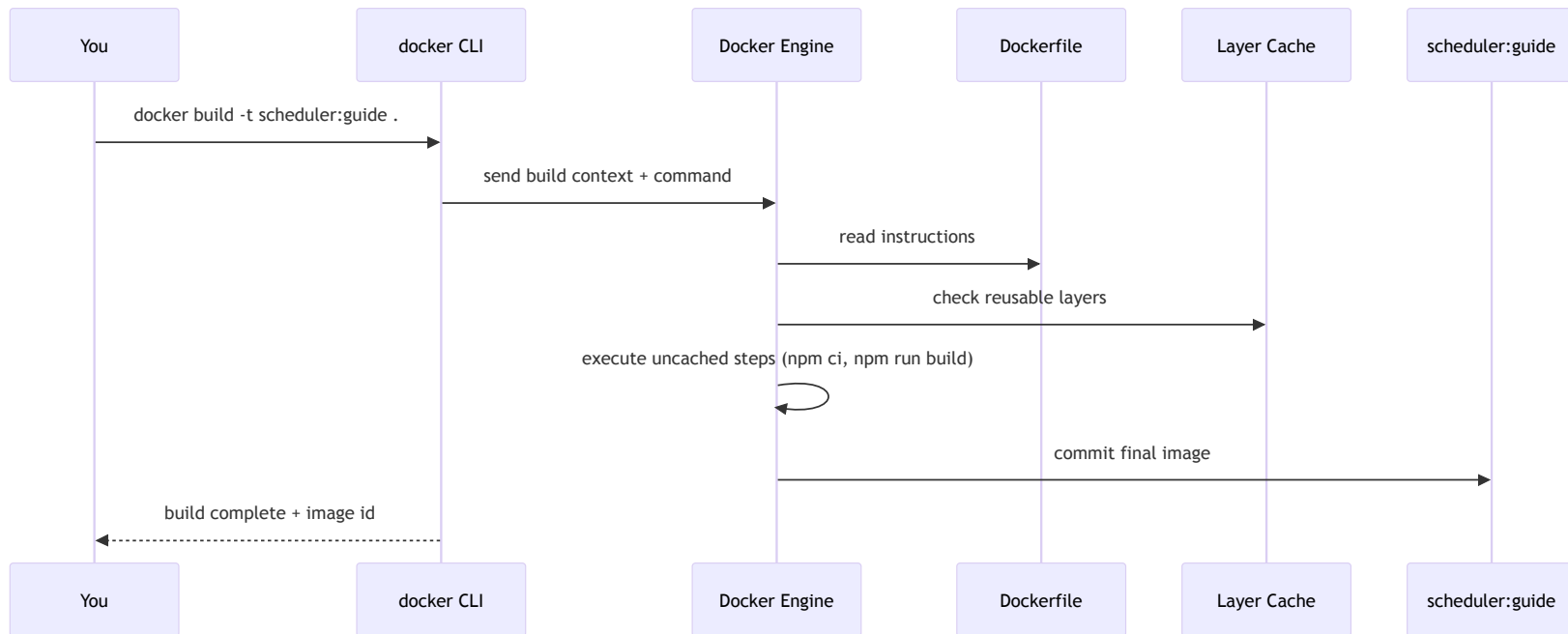
- **Build context** is everything under `.` – keep it small with `.dockerignore` .

Demo: Build the Image

Watch the build steps run and observe cached layers on rebuilds.

```
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
bash-3.2$
```

Build Sequence (Conceptual)



Takeaway:

- Docker executes the Dockerfile step-by-step

Image Inventory

```
1 docker image ls
2 docker image ls scheduler:guide
```

Look for:

- Repository + tag
- Image ID
- Size

You can use the tag or the image ID to reference the image in commands.

Example Output

1	REPOSITORY	TAG	IMAGE ID	CREATED
2	scheduler	guide	1a2b3c4d5e6f	2 minutes ago

Headers explained:

- **REPOSITORY:** image name (e.g., `scheduler`)
- **TAG:** version label (e.g., `guide` , `latest`)
- **IMAGE ID:** unique image identifier
- **CREATED:** how long ago the image was built
- **SIZE:** disk space used by the image

Run the App Container

```
1  docker run --name scheduler-guide --env-file .env -p 3000:3000 scheduler:guide
```

Flags:

- `--name` : stable container name
- `--env-file .env` : load env vars
- `-p 3000:3000` : host → container

Important:

- `.env` stays local (don't commit secrets)
- `-p` is required to access the app from your browser

Port Mapping Explained

`-p host:container`

- Host `3000` forwards to container `3000`
- Browser hits `localhost:3000`
- Container app listens on `3000`

Key points:

- Port mapping is **explicit** – without `-p`, the app is isolated
- You can change the host port (`-p 3001:3000`) if 3000 is busy
- The container still listens on its internal port (`3000`)

Flow

Browser → Host 3000 → Docker Engine → Container 3000

Think of this like a map key in data structures: the host port is the “public key” you look up, and Docker forwards (maps) it to the container’s private port where the app actually lives.

In practice, this is why you can run multiple containers of the same app on a single machine, each one can keep its internal port (3000) while you choose different host ports (3001, 3002, 3003) to reach them.

This turns out to be extremely useful when you start running multiple instances of the same service for load testing or scaling. We will look at this later.

Validate the App

Validate the app is running by visiting:

- http://localhost:3000
- http://localhost:3000/health

Health Response:

```
1 { "ok": true }
```

This response is short, but meaningful. In real systems, the response would include more details (e.g., database status), but the key is that it's a **machine-readable check** that says "this service is alive."

Why `/health` matters:

- It's a **machine-readable check** that says "this service is alive."
- In multi-service systems, orchestrators and load balancers use health checks to decide which containers should receive traffic.
- If a container fails its health check, it can be restarted automatically.

If `/health` fails, check logs and port mapping first.

Container Lifecycle

```
1  docker stop scheduler-guide
2  docker start scheduler-guide
3  docker rm scheduler-guide
```

One-step force remove:

```
1  docker rm -f scheduler-guide
```

Tip: use `stop` + `rm` in normal workflows; `rm -f` is for quick resets.

Inspect Running State

```
1 docker ps
2 docker ps -a
```

Use:

- `ps` : running containers
- `ps -a` : all containers

Read the Columns

- CONTAINER ID
- IMAGE
- STATUS
- PORTS
- NAMES

Use the container **name** for logs/inspect/exec commands.

View Logs

```
1 docker logs scheduler-guide
2 docker logs -f scheduler-guide
```

When to use:

- `logs` : quick snapshot
- `logs -f` : live stream while testing

Tip: stop `-f` with `Ctrl+C` (container keeps running).

Inspect Metadata

```
1  docker inspect scheduler-guide
2
3  docker inspect scheduler-guide --format '{{.State.Status}}'
4  docker inspect scheduler-guide --format '{{json .NetworkSettings.Ports}}'
```

Use to confirm:

- container state
- port publishing
- mounts + env

The formatted `inspect` is the fastest way to verify port bindings.

Exec Into the Container

```
1  docker exec -it scheduler-guide sh
```

Inside, useful checks:

```
1  pwd
2  ls
3  cat package.json
4  ls dist
5  exit
```

Tip: `sh` works for Alpine images; use `bash` only if installed.

Process & Resource Visibility

```
1 docker top scheduler-guide
2 docker stats scheduler-guide
```

Use:

- `top` : processes
- `stats` : CPU/memory/network

Watch for unexpected processes or CPU spikes.

Example Snippet

1	CONTAINER ID	NAME	CPU %	MEM USAGE
2	1a2b3c4d5e6f	scheduler-guide	0.05%	45MiB / 7.1GiB

Debugging Workflow (Do This Order)

1. Is it running?

- `docker ps -a`

2. If exited, why?

- `docker logs <name>`

3. Port published?

- `docker inspect <name> --format '{{json .NetworkSettings.Ports}}'`

4. App healthy?

- `http://localhost:3000/health`

5. Need deeper check?

- `docker exec -it <name> sh`

This flow fixes most “container won’t start” issues quickly.

Cleanup Commands (Use Carefully)

```
1 docker container prune
2 docker image prune
3 docker system prune -a
```

Warnings:

- `prune` deletes unused resources
- `system prune -a` is aggressive

Always read the prompt before confirming a prune.

When to Use

- After a tutorial/lab
- When disk usage grows
- Never when you need old images

Common Mistakes & Fixes

Port is already allocated

```
1  docker run --name scheduler-guide --env-file .env -p 3001:3000 scheduler:guide
```

Cannot connect to the Docker daemon

- Start Docker Desktop, retry `docker info`

No such image

- Run `docker image ls` to confirm tags

Tip: image names are `repository:tag` (e.g., `scheduler:guide`).

Mini Lab (Proficiency Check)

1. Build image `scheduler:lab`
2. Run `scheduler-lab` on `3000:3000`
3. Verify `/health`
4. `docker exec -it` into container
5. Follow logs with `docker logs -f`
6. Stop and remove container
7. Remove image

If you can do this from memory, you're Docker-ready.

Bonus: repeat with a different tag to prove you understand image naming.

Résumé-Ready Summary

You can now:

- Build and tag images
- Run containers with env + ports
- Inspect logs, state, ports
- Debug with `inspect` + `exec`
- Clean up safely

Next Step

- Docker Compose
- Multi-container systems
- Service dependencies

Compose is the bridge from single-container to real systems.

Appendix: Useful Commands (Quick Reference)

```
1  # Build + run
2  docker build -t scheduler:guide .
3  docker run --name scheduler-guide --env-file .env -p 3000:3000 scheduler:guide
4
5  # Inspect
6  docker ps
7  docker logs -f scheduler-guide
8  docker inspect scheduler-guide --format '{{.State.Status}}'
9  docker exec -it scheduler-guide sh
10
11 # Cleanup
12 docker stop scheduler-guide
13 docker rm scheduler-guide
14 docker image rm scheduler:guide
```